FAKULTA
INFORMAČNÍCH
TECHNOLOGIÍ
ČVUT V PRAZE

NI-NLM – Lecture 2

Tokenization & the Transformer architecture

Zdeněk Kasner

24 Feb 2026

Recap & math refresher

Recap from last lecture

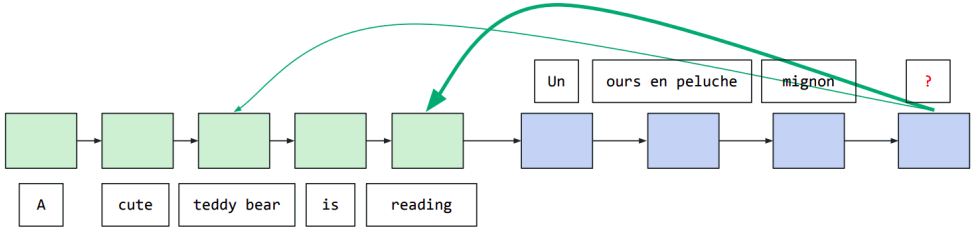
Last time, we covered:

- How to represent **language**?
- What is language **modeling**?
- How to process language with **neural networks**?
  - Also how to do language modeling with neural networks.

Today, we will cover:

How to process language in a more **efficient and scalable way**?

(→ tokenization, attention, Transformer)



source: Stanford CME295

Matrix multiplication

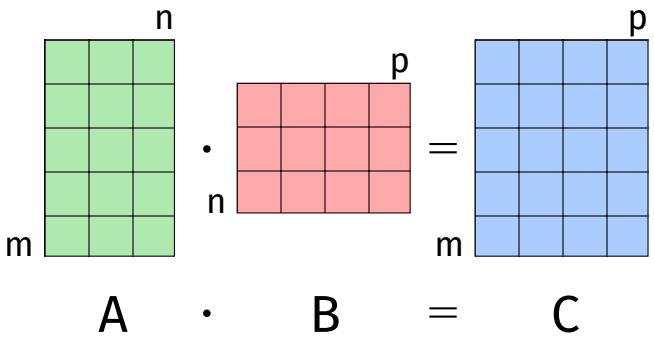
Given matrices $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ and $\boldsymbol{B} \in \mathbb{R}^{n \times p}$, their product $\boldsymbol{C} = \boldsymbol{AB}$ is:

$$C_{i,j} = \sum_{k=1}^{n} A_{i,k} B_{k,j}$$

Example: matrix-vector multiplication:

$$\begin{pmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{pmatrix} \cdot \begin{pmatrix} b_1 \\ b_2 \end{pmatrix} = \begin{pmatrix} a_{1,1} b_1 + a_{1,2} b_2 \\ a_{2,1} b_1 + a_{2,2} b_2 \end{pmatrix}$$

$$\underbrace{(2 \times 2)}_{\boldsymbol{A}} \cdot \underbrace{(2 \times 1)}_{\boldsymbol{b}} = \underbrace{(2 \times 1)}_{\boldsymbol{c}}$$



source: Wikipedia

Single-layer feed-forward neural network

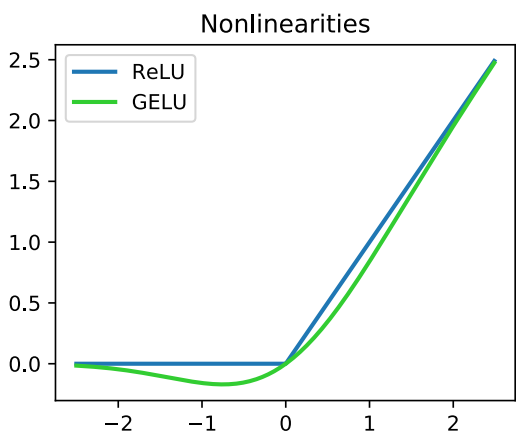
A single-layer **feed-forward neural network** (FFNN) applies a linear transformation (using the matrix $\boldsymbol{W}$ and bias $\boldsymbol{b}$) followed by a non-linear function σ (ReLU, GELU, ...):

$$\boldsymbol{y}_1 = \sigma(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1)$$

Example: FFNN with ReLU applied on a vector $\boldsymbol{x}$:

$$\underbrace{\begin{pmatrix} -0.38 & 1.35 \\ 0.70 & 0.30 \\ -1.03 & -1.03 \end{pmatrix}}_{\boldsymbol{W}_1 \in \mathbb{R}^{3\times 2}} \cdot \underbrace{\begin{pmatrix} -0.88 \\ 0.73 \end{pmatrix}}_{\boldsymbol{x} \in \mathbb{R}^2} + \underbrace{\begin{pmatrix} 0.10 \\ 0.21 \\ -0.48 \end{pmatrix}}_{\boldsymbol{b}_1 \in \mathbb{R}^3} = \underbrace{\begin{pmatrix} 1.42 \\ -0.19 \\ -0.33 \end{pmatrix}}_{\boldsymbol{z} \in \mathbb{R}^3}$$

$$\mathrm{ReLU}\left(\begin{pmatrix} 1.42 \\ -0.19 \\ -0.33 \end{pmatrix}\right) = \underbrace{\begin{pmatrix} 1.42 \\ 0.00 \\ 0.00 \end{pmatrix}}_{\boldsymbol{y}_1 \in \mathbb{R}^3}$$



source: Wikipedia

Softmax

Given a vector $x \in \mathbb{R}^d$, **softmax** ensures that $\forall x_i \geq 0$ and $\sum(x) = 1$:

$$\mathrm{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^{K} \exp(x_j)}$$

How to think about softmax

A convenient way how to transform any vector to a valid probability distribution.

Example:

$$z = [2.0, 1.0, 0.1]$$

$$\mathrm{softmax}(z) = [0.659, 0.243, 0.099]$$

Implementation in numpy:

```
def softmax(x):
    return np.exp(x) / np.sum(np.exp(x))
```

Why softmax?

Given a vector $x \in \mathbb{R}^d$, **softmax** ensures that $\forall x_i \geq 0$ and $\sum(x) = 1$:

```
def softmax(x):
    return np.exp(x) / np.sum(np.exp(x))
```

However, the same can be achieved with these two normalization functions:

```
def l1_norm(x):
    return np.abs(x) / np.sum(np.abs(x))
```

```
def min_sum_norm(x):
    x_norm = (x - np.min(x))
    return x_norm / np.sum(x_norm)
```

Question

What are the specifics and (dis)advantages of each of these three functions?

Tokenization

From words to tokens

Tokenization = splitting text into smaller units (**tokens**) that the model will process.

We assumed that our input is a **sequence of words**. That sounds good in theory, but worse in practice:

Question

How would you split these strings into meaningful units?

- `Pneumonoultramicroscopicsilicovolcanoconiosis`
- 大型语言模型
- 🤗❤️🍾🥳

Why subword tokenization?

Idea

Keep the common words, split rare words into subwords.

Compare:

- Word-level: `["unhappiness"]` → out of vocabulary
- Character-level: `["u","n","h","a","p","p","i","n","e","s","s"]`
- Subword: `["un", "happiness"]` ✓

Question

How to decide where to split the subwords?

Byte pair encoding (BPE)

source: Sennrich et al. 2016

Originally a data compression algorithm, adapted for NLP tokenization.

BPE algorithm

1. Start with a vocabulary of all **individual characters** in the training corpus.
2. Count all **adjacent pairs** of tokens in the corpus.
3. **Merge** the most frequent pair into a new token.
4. **Repeat** steps 2–3 until the desired vocabulary size is reached.

The result is a **merge table** that defines how to tokenize any new text.

📝 **Exercise time!**

BPE – step by step

Corpus: "aab baac aab baac caaba aac" → initial vocabulary: {a, b, c, _}

Step	Most frequent pair	New token
1	(a, a) → 6 times	aa
2	(aa, b) → 3 times	aab
3	(aa, c) → 3 times	aac
4	(aab, _) → 2 times	aab_

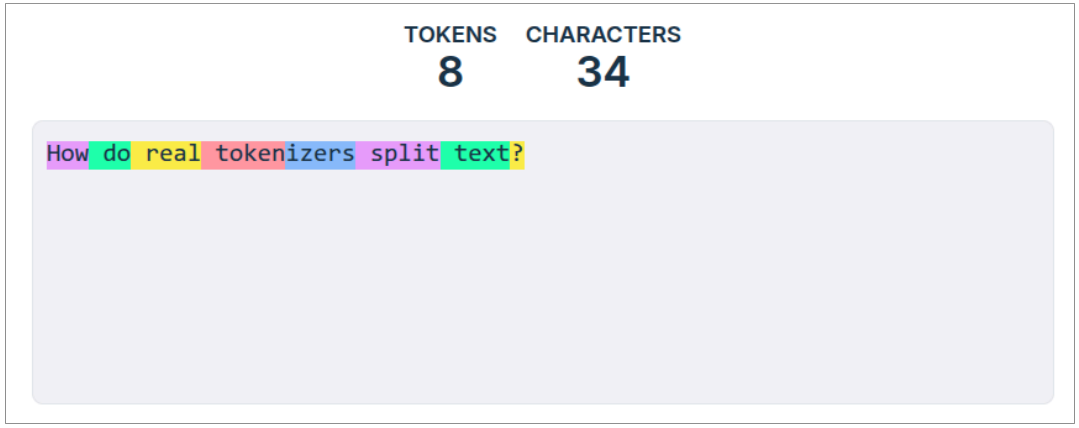
Final vocabulary: {_, a, b, c, aa, aab, aac, aab_}

Typical vocabulary sizes in practice: **50k–200k** tokens.

Tokenizer playground

How do real tokenizers split text?

👉 https://huggingface.co/spaces/Xenova/the-tokenizer-playground



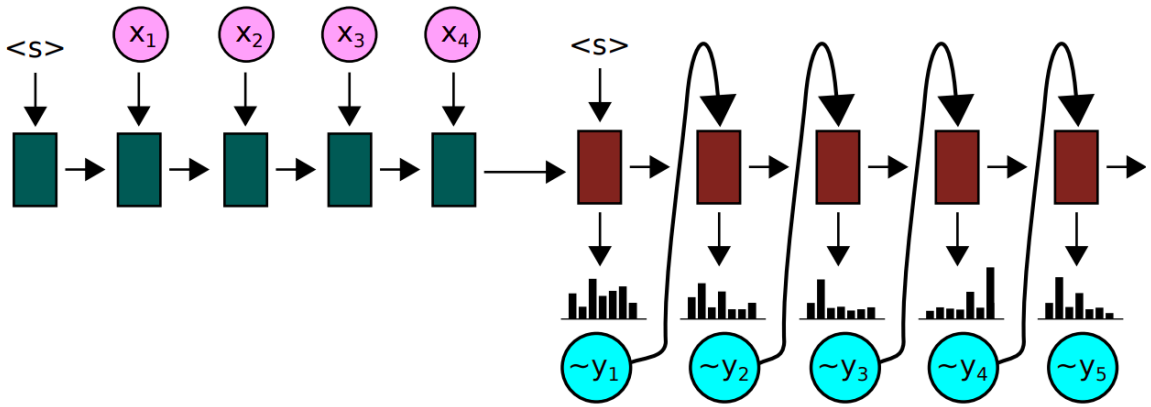
Notice the difference between languages.

Attention mechanism

The seq2seq bottleneck

Remember: the encoder compresses the **entire input** into a single vector $\boldsymbol{h}_N$.

- Works OK for short sequences, but for longer inputs the **information gets lost**.
- The decoder has no way to "look back" at specific parts of the input.



source: NPFL116

Attention – the idea

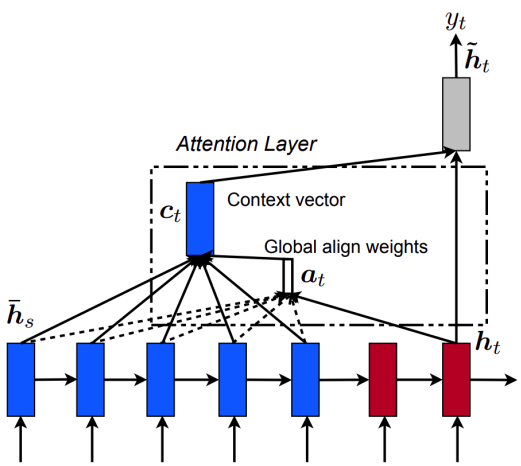
Idea

Let the decoder **pick the information from the encoder hidden states**.

source: Luong et al. (201

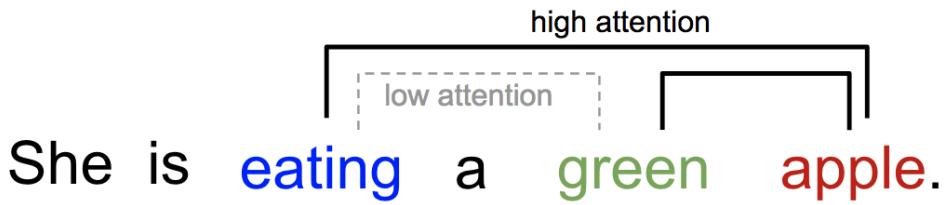
How exactly? At each decoding step t:

1. Compute a **score** for each encoder state $\boldsymbol{h}_i$.
2. Normalize scores with **softmax** → attention weights $\alpha_{t,i}$.
3. Compute a **weighted sum** of hidden states → context vector: $\boldsymbol{c}_t = \sum_i \alpha_{t,i} \boldsymbol{h}_i$.
4. Use $\boldsymbol{c}_t$ together with the decoder state to predict the next token.

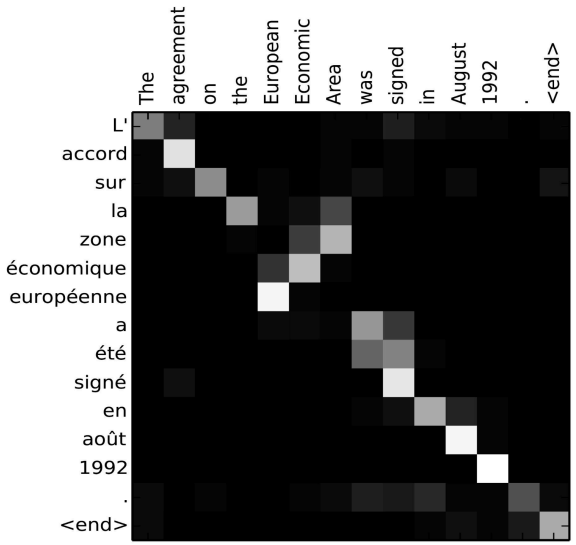


Attention with RNNs

The attention weights correspond to **which input tokens the decoder focuses on** at each step.



source: Lil'Log



source: Bahdanau et al. (2014)

But it is slow...

RNN needs to process the input sequentially, token-by-token → there is no way to parallelize the process.

Idea

Can we get rid of the RNN entirely and use **only attention**?

The Transformer

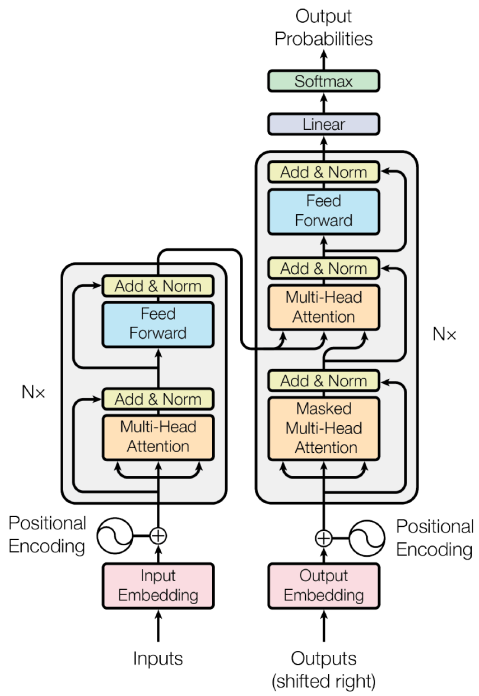
Attention is all you need

source: Vaswani et al. 2017

Paper by Google in 2017, originally proposed for improving MT systems.

Key ideas:

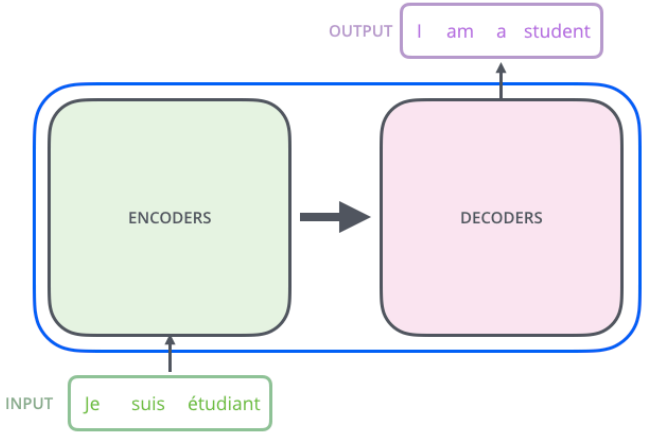
- Process each state **in parallel with FFNNs**.
- Use the **attention mechanism** to share information between the tokens.
- Apply it repeatedly in **layers** to make it more expressive.
- Use **positional encoding** to inject position information.



Transformer – high-level view

source: The Illustrated Transformer

The (original!) Transformer follows the **encoder-decoder** pattern (→seq2seq).



- **Encoder**: reads the input sequence, produces a rich representation.
- **Decoder**: generates the output sequence one token at a time.

Title

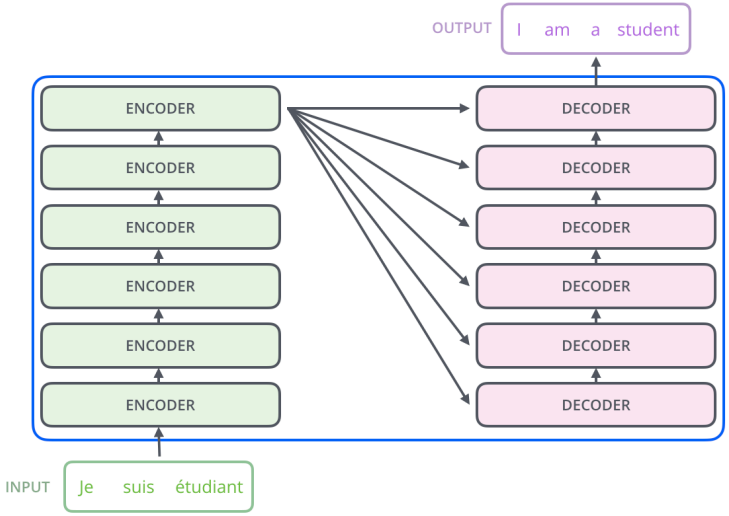
TODO - what transformer does, explained once again

The encoder-decoder stack

source: The Illustrated Transformer

Both encoder and decoder are **stacks of identical layers** (called “blocks”).

- The original Transformer uses 6 layers for both.
- Each layer refines the representation.
- The output of the last encoder layer is passed to **every decoder layer**.



Current scales

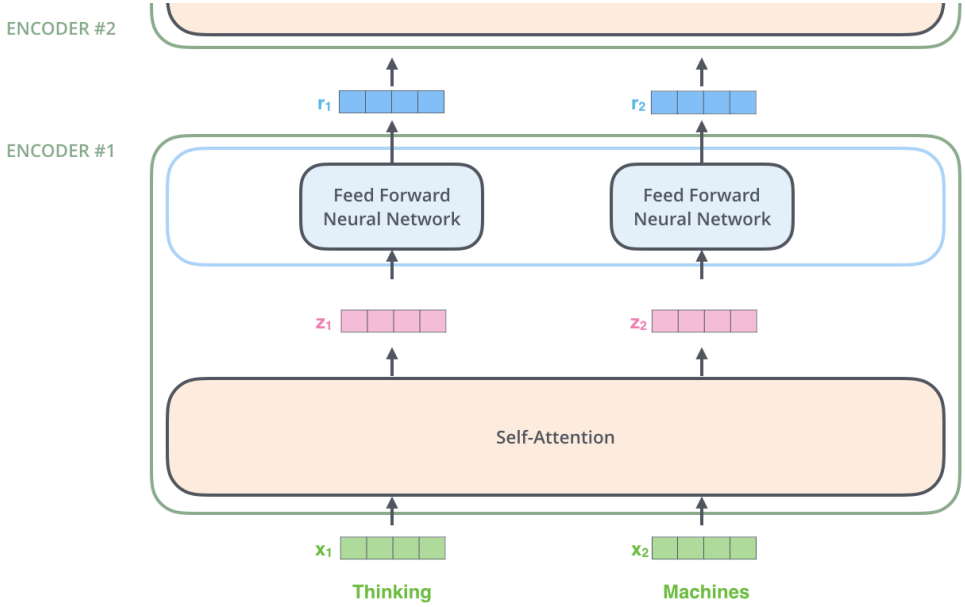
Modern LLMs use many more layers: e.g. Llama 3.3 70B has **80 (decoder) layers**.

Inside an encoder block

source: The Illustrated Transformer

Each encoder block has two sub-layers:

1. **Self-attention layer** → sharing information between the tokens.
2. **Feed-forward layer** → updating the token information.



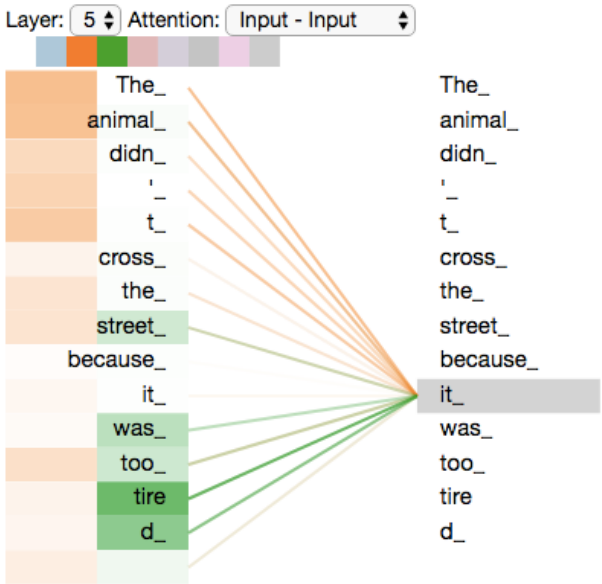
Self-attention

source: The Illustrated Transformer

The original attention mechanism was useful for decoding a new sequence.

But what if we only want to **represent a sequence**?

→ We can turn it into **self-attention**.



Example

*"The animal didn't cross the street because **it** was too tired."*

When processing *"it"*, the model should attend strongly to *"animal"* (not *"street"*).

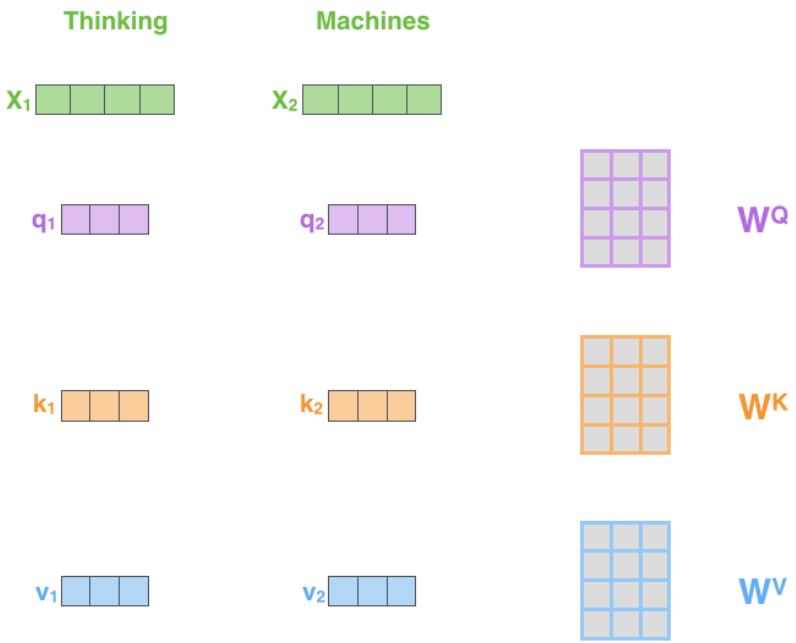
Self-attention – query, key, value

source: The Illustrated Transformer

For each token embedding x_i, we compute three vectors:

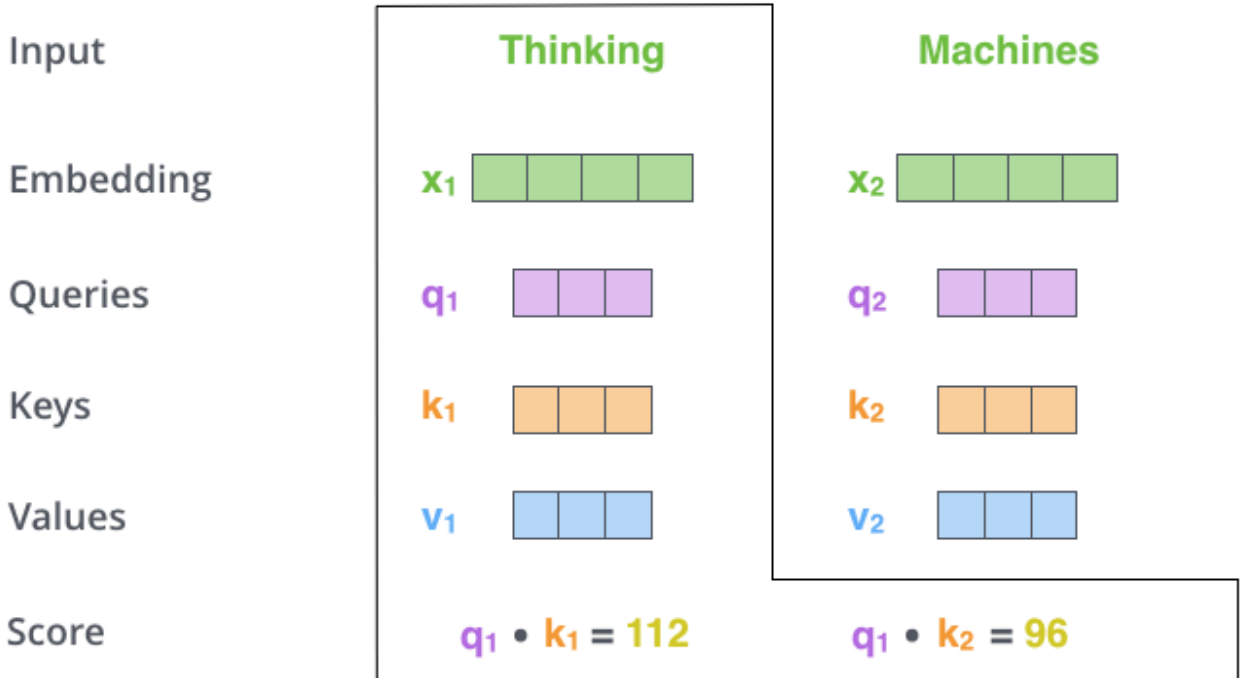
$$q_i = W^Q x_i, \quad k_i = W^K x_i, \quad v_i = W^V x_i$$

- **Query** (q): "what am I looking for?"
- **Key** (k): "what do I contain?"
- **Value** (v): "what information do I provide?"



Self-attention – computation

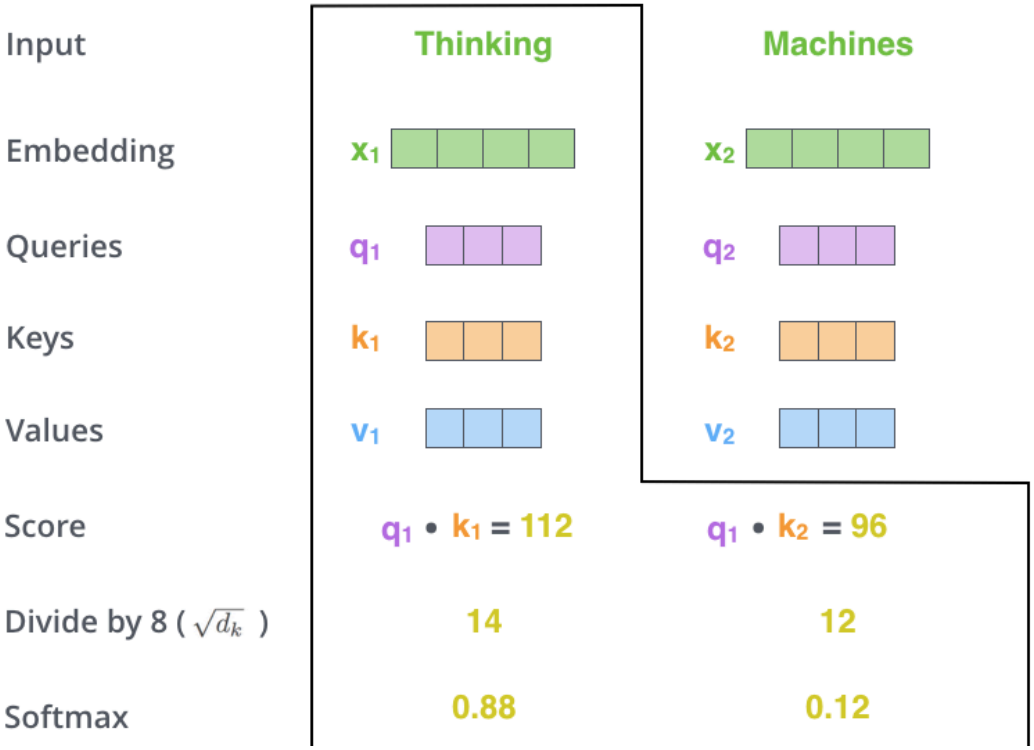
We multiply queries q_i by keys k_i → **raw (non-normalized) attention scores.**



Self-attention – computation

source: The Illustrated Transformer

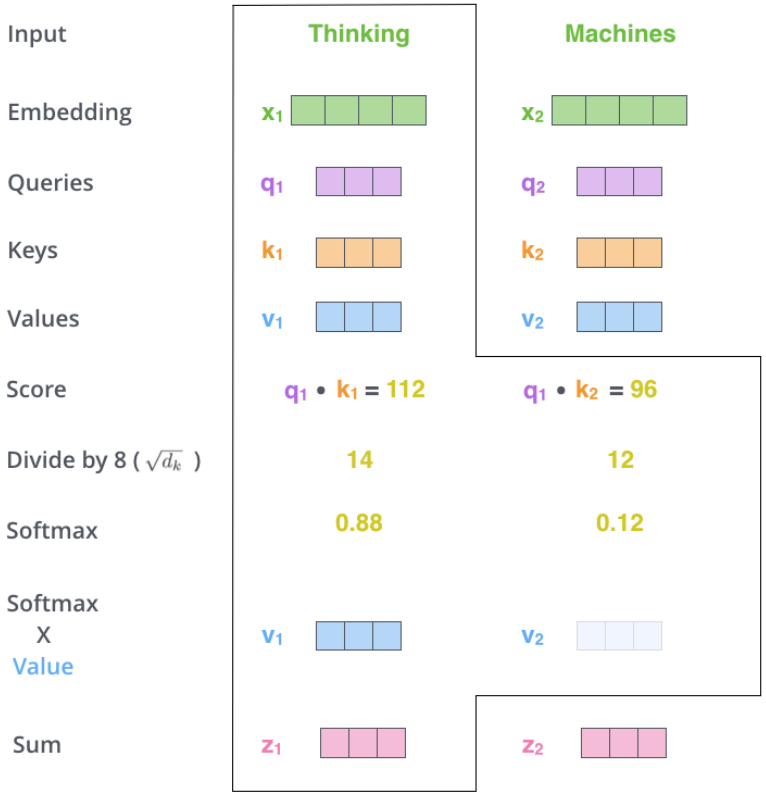
We normalize the scores: divide by $\sqrt{d_k}$ and apply `softmax` → **attention weights**.



Self-attention – computation

source: The Illustrated Transformer

The output for token i is a vector z_i: **weighted mixture** of all value vectors.



Pause and ponder

Why do we need all three?

Self-attention – computing scores

In practice, we compute everything in **one matrix operation**:

$$\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V}$$

where $\boldsymbol{Q} \in \mathbb{R}^{n \times d_k}$, $\boldsymbol{K} \in \mathbb{R}^{n \times d_k}$, $\boldsymbol{V} \in \mathbb{R}^{n \times d_v}$.

source: Attention is All You Need Formula T-Shirt

Multi-head attention

source: Vaswani et al. 2017

A single attention head captures **one type of relationship**. But words relate to each other in many ways: syntactic, semantic, positional, ...

Question

How would you capture these relationships?

Idea

Run **multiple attention heads in parallel**, each with its own $\boldsymbol{W}^Q$, $\boldsymbol{W}^K$, $\boldsymbol{W}^V$, let each head learn a different pattern.

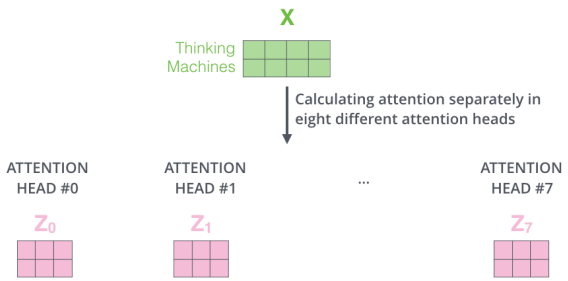
Multi-head attention

source: The Illustrated Transformer

With h heads and size of the hidden state d:

- Each head uses $d_k = d_v = \frac{d}{h}$.
- Each head computes its own Q, K, V and produces an output $\boldsymbol{Z}_i$.

$$\text{head}_i = \text{Attention}(\boldsymbol{Q}_i, \boldsymbol{K}_i, \boldsymbol{V}_i)$$



source: Lil'Log

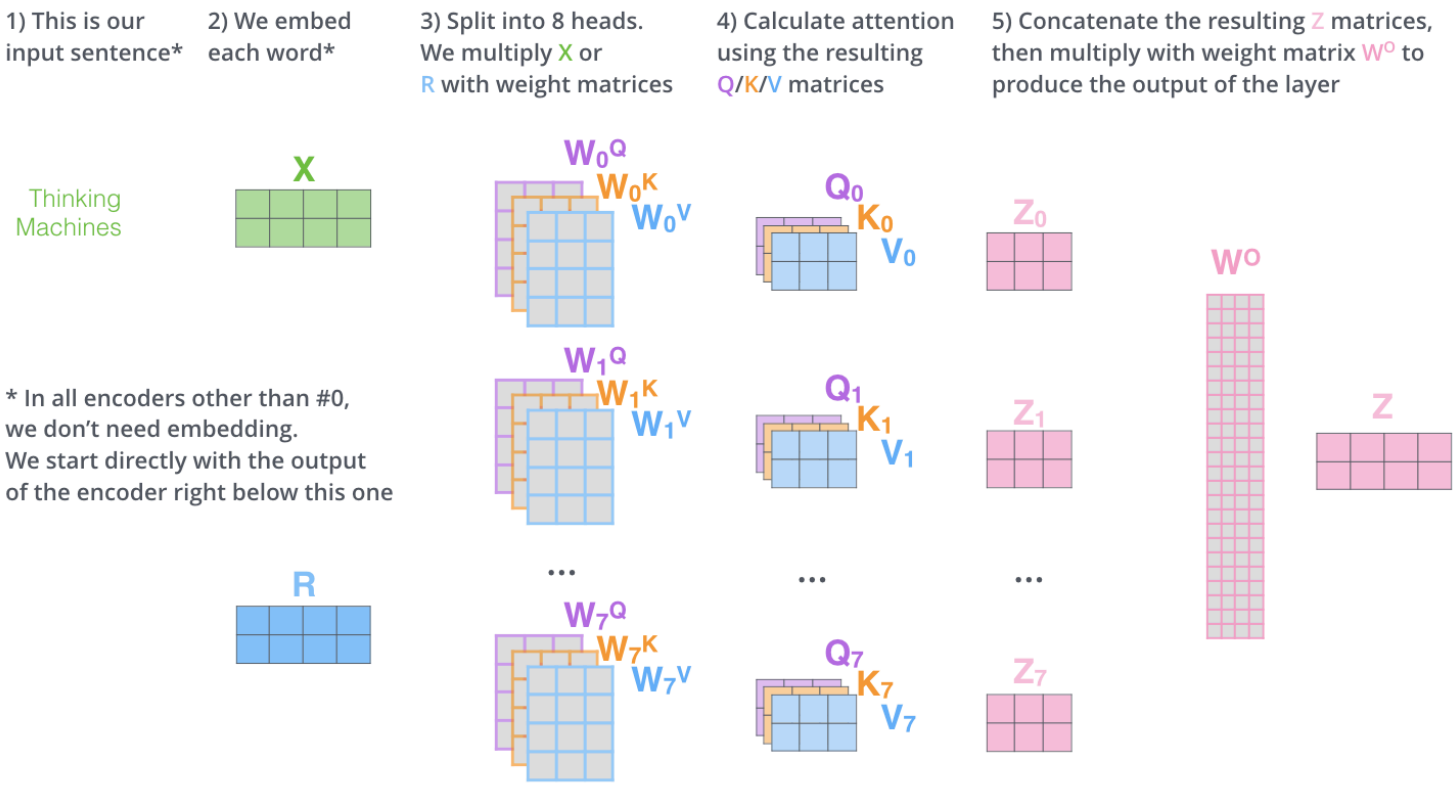
The heads are **concatenated** and projected:

$$\text{MultiHead} = [\boldsymbol{Z}_1; ...; \boldsymbol{Z}_h]\boldsymbol{W}^O$$

where $\boldsymbol{W}^O \in \mathbb{R}^{hd_v \times d}$.

Self-attention: putting it all together

source: The Illustrated Transformer



Residual connections & layer normalization

source: The Illustrated Transformer

Each sub-layer (attention or FFN) is wrapped in:

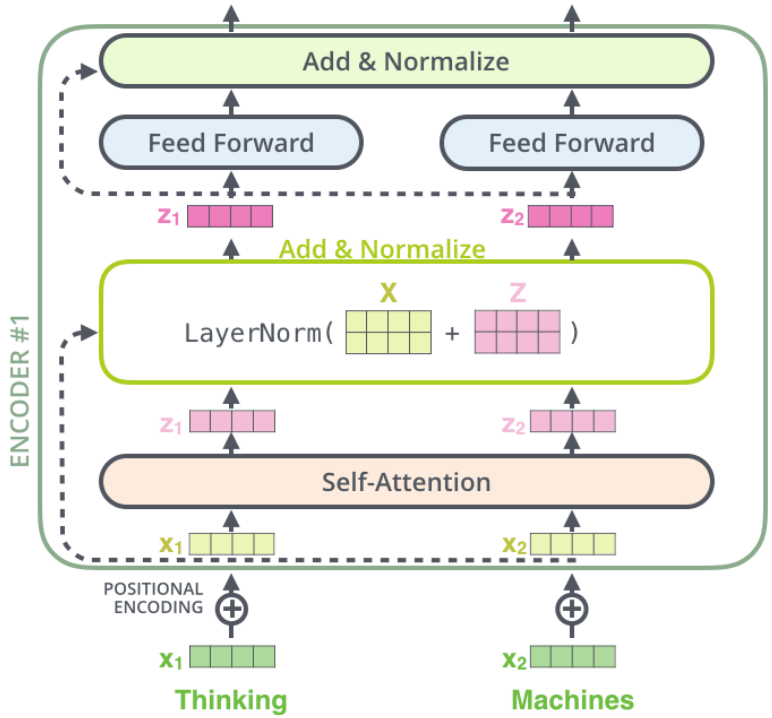
$$\mathrm{LayerNorm}(\boldsymbol{x} + \mathrm{SubLayer}(\boldsymbol{x}))$$

Residual connection ($x + \ldots$):

- Helps gradients flow through deep networks.
- The layer only needs to learn the **"delta"**.

Layer normalization:

- Normalizes across features (not across batch).
- Stabilizes training.



Position-wise feed-forward network

Applied to **each token independently** (same weights, different inputs):

$$\mathrm{FFN}(\boldsymbol{x}) = \boldsymbol{W}_2 \cdot \mathrm{ReLU}(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1) + \boldsymbol{b}_2$$

where $\boldsymbol{W}_1 \in \mathbb{R}^{d \times d_{\mathrm{ff}}}$, $\boldsymbol{W}_2 \in \mathbb{R}^{d_{\mathrm{ff}} \times d}$.

Dimensions

In the original Transformer:

- $d = 512$ (model dimension)
- $d_{\mathrm{ff}} = 2048$ (inner dimension, 4× larger)

What does the FFN do?

The FFN processes each token **on its own**, acting as a **per-token memory** storing factual knowledge.

Positional encoding – why do we need it?

Self-attention is **permutation-invariant**: if we shuffle the input tokens, the attention scores would be the same (up to the same permutation).

Question

Why is that a problem? *"The cat sat on the mat"* and *"mat the on sat cat the"* should be different!

Idea

We need to **inject position information** into the input embeddings so that the model can distinguish token order.

Positional encoding – sinusoidal

source: Vaswani et al. 2017

The original Transformer uses **fixed sinusoidal functions**, added to the token embeddings:

$$\boldsymbol{x}_i = \mathrm{Embed}(w_i) + \mathrm{PE}(i)$$

$$\mathrm{PE}(p, 2i) = \sin\left(\frac{p}{10000^{2i/d}}\right), \quad \mathrm{PE}(p, 2i+1) = \cos\left(\frac{p}{10000^{2i/d}}\right)$$

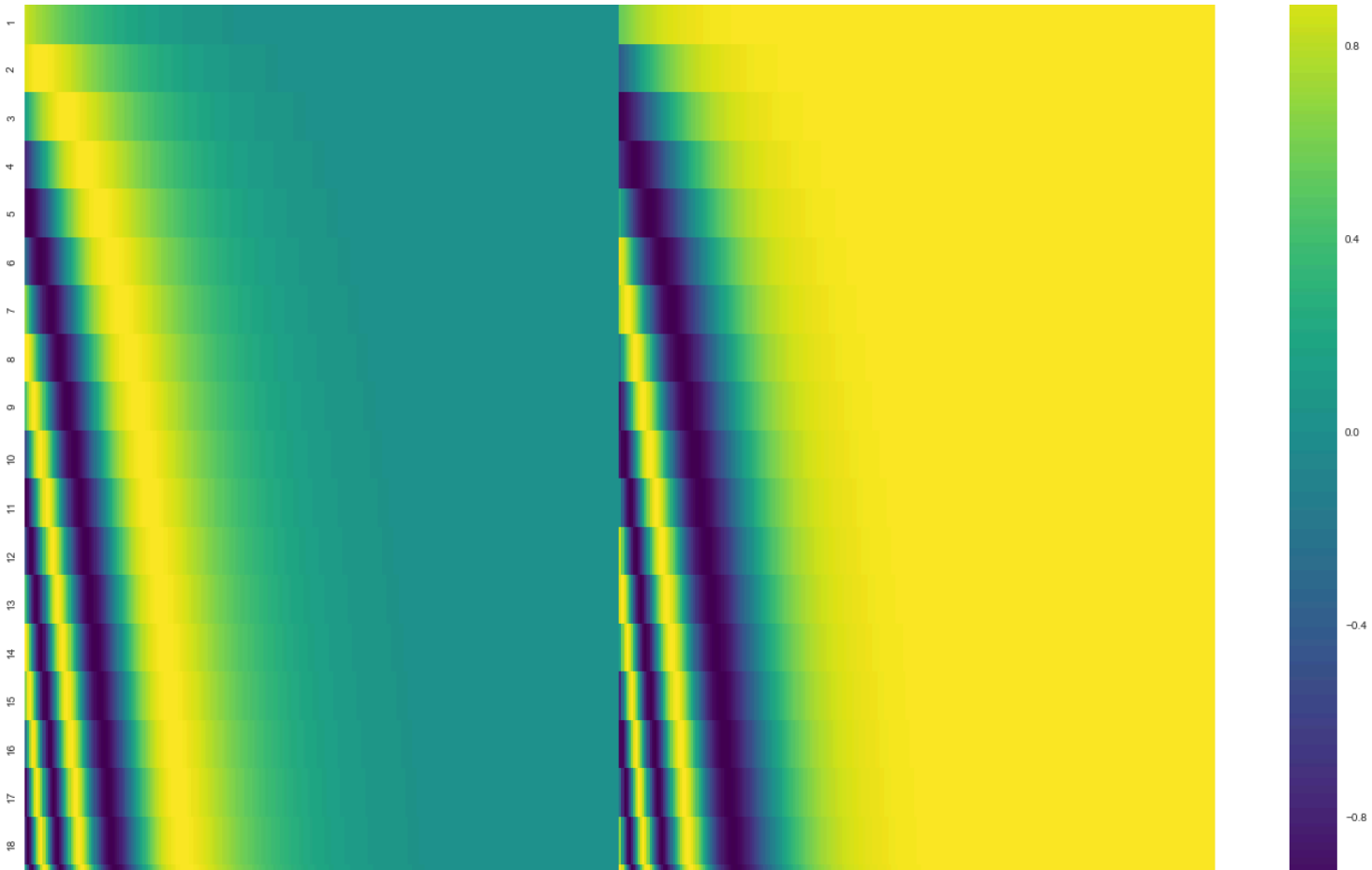
where p is the position and i is the dimension index.

- Each position gets a **unique encoding**.
- Nearby positions have **similar encodings**.
- The model can learn to attend to **relative positions**.

Modern models often use **learned** positional embeddings or **rotary positional encoding** (RoPE) instead.

Positional encoding – visualization

source: The Illustrated Transformer



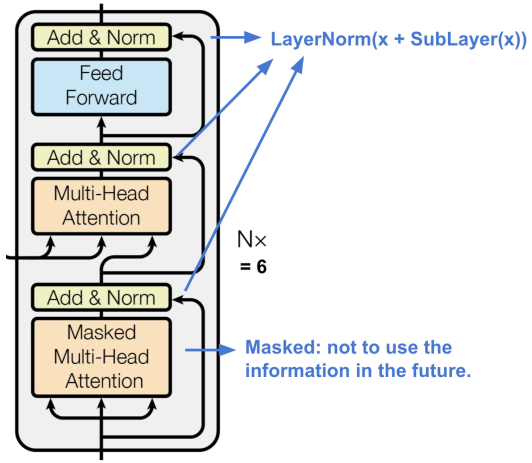
Each row is a positional encoding vector. The pattern changes across dimensions: low-frequency sinusoids on the left, high-frequency on the right.

The decoder

source: The Illustrated Transformer

The decoder has **three sub-layers** per block:

1. **Masked self-attention**
   - Each token can only attend to previous tokens (no peeking into the future).
2. **Encoder-decoder (cross) attention**
   - Queries come from the decoder, keys and values come from the encoder output.
3. **FFN**
   - Same as in the encoder.



source: Lil'Log

Masked self-attention

source: Vaswani et al. 2017

During training, we know the full target sequence. But we must prevent the decoder from **looking ahead**.

We apply a **mask** to the attention scores before softmax:

$$\mathrm{MaskedAttention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}} + \boldsymbol{M}\right)\boldsymbol{V}$$

where $\boldsymbol{M}$ is a matrix with 0 for allowed positions and $-\infty$ for future positions.

Effect

$\exp(-\infty) = 0$ → after softmax, future tokens get zero attention weight. Token t can only attend to tokens $1, \ldots, t$.

Encoder-decoder attention

In the cross-attention sub-layer:

- **Queries** come from the decoder (previous sub-layer).
- **Keys** and **values** come from the **encoder output**.

$$\mathrm{CrossAttention}(\boldsymbol{Q}_{\mathrm{dec}}, \boldsymbol{K}_{\mathrm{enc}}, \boldsymbol{V}_{\mathrm{enc}})$$

This allows every decoder position to attend over **all positions in the input** – just like the attention mechanism in the RNN seq2seq, but more powerful.

> **This is the bridge**

> Cross-attention is what connects the encoder and decoder. Without it, the decoder would have no access to the source sequence.

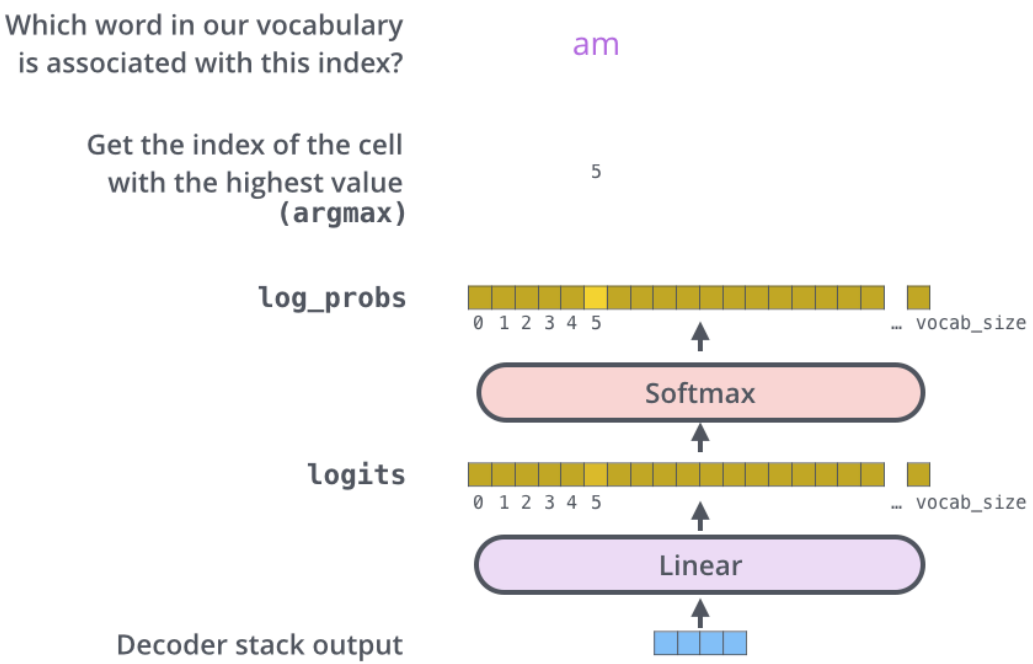
Output layer

source: The Illustrated Transformer

The final decoder output is projected to vocabulary size and passed through **softmax**:

$$P(w_t \mid w_1, \ldots, w_{t-1}, \boldsymbol{x})$$
$$= \mathrm{softmax}(\boldsymbol{W}_{\mathrm{out}} \boldsymbol{h}_t + \boldsymbol{b})$$

Same idea as neural language modeling we talked about the last time – but now the hidden state $\boldsymbol{h}_t$ is much richer thanks to attention.



Transformer vs. RNNs

RNNs

- Sequential processing → **slow**.
- Information must pass through **all intermediate states**.
- Gradient vanishing over long distances.
- Hidden state is a **bottleneck**.

Transformer

- Parallel processing → **fast**.
- Any token can directly attend to **any other token** (path length = 1).
- Stable gradients with residual connections.
- No bottleneck: information distributed across **all positions**.

Summary

Summary

What we covered today

- **Refresher**: matrix multiplication, FFNNs, softmax.
- **Tokenization**: subword tokenization with BPE.
- **Attention**: solving the seq2seq bottleneck (Bahdanau, Luong).
- **The Transformer**:
  - Self-attention with Q, K, V → scaled dot-product attention.
  - Multi-head attention → multiple parallel attention heads.
  - Positional encoding → injecting position information.
  - Residual connections + layer normalization.

Links

- The Illustrated Transformer – Jay Alammar
- Attention? Attention! – Lil'Log
- The Annotated Transformer – Harvard NLP
- Attention is all you need – Vaswani et al. 2017
- Neural machine translation by jointly learning to align and translate – Bahdanau et al. 2015
- Neural machine translation of rare words with subword units – Sennrich et al. 2016
- Tokenizer playground – Hugging Face